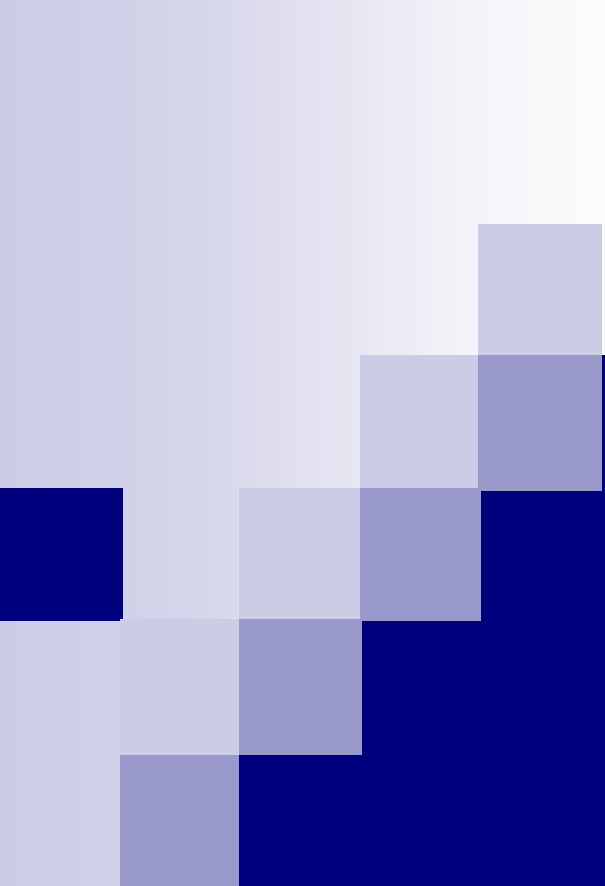




Software Configuration Management – 1

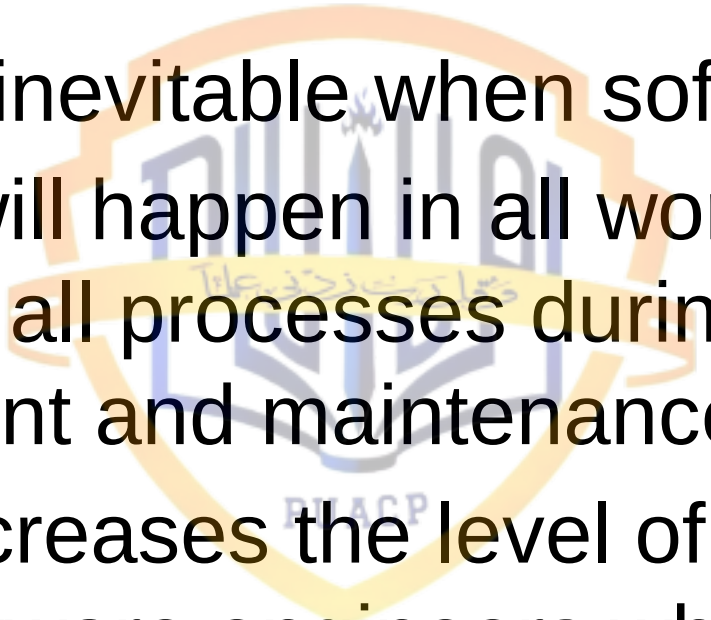
Lecture # 33

Recap

- In the last several lectures, we have talked about software testing
- Software testing is an important and integral process in software development and often the only mechanism for software quality assurance

Today's Lecture

- Starting today, we'll move to another important area critical to software quality assurance
- This area is software configuration management
- In today's lecture, we'll introduce the basic concepts of software configuration management and develop a foundation for further discussions in later lectures

- 
- 
- Change is inevitable when software is built
 - Changes will happen in all work products and during all processes during software development and maintenance
 - Change increases the level of confusion among software engineers who are working on a software project

Confusion

- Confusion arises when changes are not
 - **Analyzed** before they are made
 - **Recorded** before they are implemented
 - **Reported** to those who need to know
 - **Controlled** in a manner that will improve quality and reduce error
- We need to minimize this confusion, or else our projects will get out of control

Configuration Management - 1

- The art of coordinating software development to minimize ...confusion is called **configuration management**



Configuration Management - 2

- Configuration management is the art of identifying, organizing, and controlling **modifications** to the software being built by a programming team
- The goal is to maximize productivity by minimizing mistakes

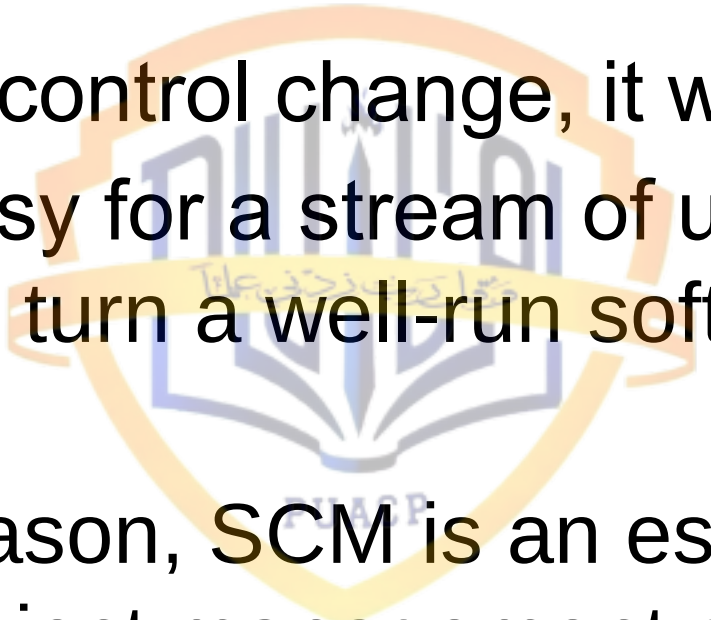
Software Configuration Management - 1

- Software configuration management (or SCM) is an umbrella activity that is applied throughout the software process
- This means that SCM is not tied to one process model or another, instead it is used in all instances of software development processes

- 
- SCM is a set of activities designed to control change by identifying the work products that are likely to change, establish relationships among them, defining mechanisms for managing different versions of these work products, controlling the changes imposed, and auditing and reporting on the changes

Purpose of SCM Activities

- Identify change
- Control change
- Ensure that the change is being properly implemented
- Report changes to others who may be interested

- 
- 
- If we don't control change, it will control us
 - It's very easy for a stream of uncontrolled changes to turn a well-run software project into chaos
 - For that reason, SCM is an essential part of good project management and is a solid software engineering practice

Software Configuration Management - 2

- So, SCM provides a cover against
 - Lack of visibility
 - Lack of control
 - Lack of traceability
 - Lack of monitoring
 - Uncontrolled change

Software Configuration Management - 3

- Software configuration management provides a means for visibility, traceability, and formally controlling the evolution of software
- SCM should be viewed as a software quality assurance activity that is applied throughout the software process
- Let's now have more detailed discussion on software configuration management

Software Configuration

- The items that comprise all information produced as part of the software process are collectively called a software configuration
 - Computer programs (source and executable)
 - Documents that describe the computer programs
 - Data
- Software configuration items will grow

Change - A Constant - 1

- The number of configuration items in a software project will grow continuously and the changes within each configuration item will also occur on a frequent basis
- So, we can say that
 - There is nothing permanent except change
 - Heraclitus (500 B.C.)

Change - A Constant - 2

- No matter where you are in the system life cycle, the system will change, and the desire to change it will persist throughout the life cycle
- Software is like a sponge due to its susceptibility to change
- We emphasize this point, because this is the first step in planning and implementing a good software configuration management process

Sources of Change - 1

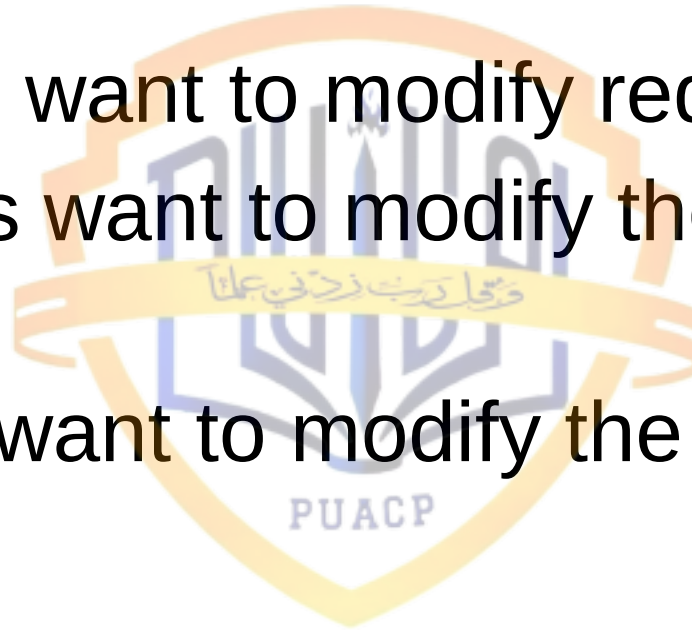
- New business or market conditions dictate changes in product requirements or business rules
- New customer needs demand modification of data produced by information systems, functionality delivered by products, or services delivered by computer-based system

Sources of Change - 2

- Reorganization or business growth / downsizing causes changes in project priorities or software engineering team structure
- Budgetary or scheduling constraints cause a redefinition of the system or product

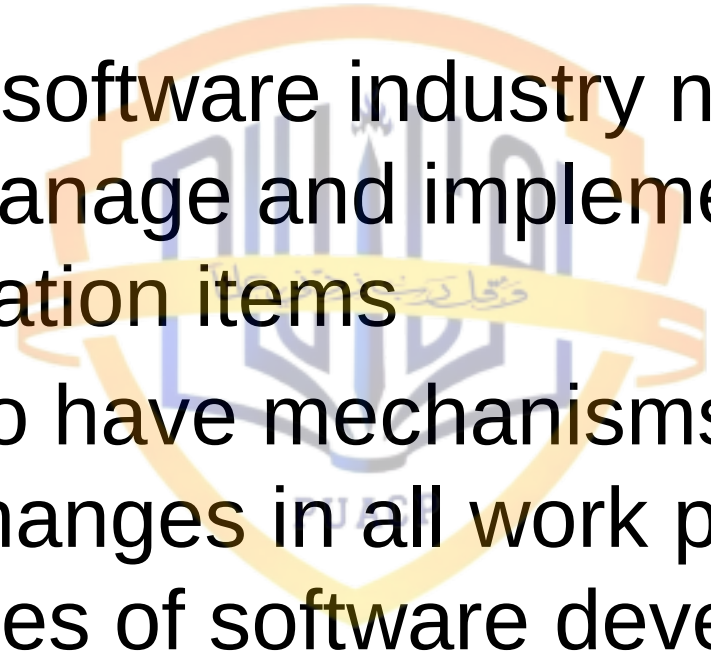
Change is Everywhere

- Customers want to modify requirements
- Developers want to modify the technical approach
- Managers want to modify the project strategy



Why All This Modification?

- As time passes, all constituencies know more
 - About what they need
 - Which approach would be best
 - How to get it done and still make money
- **Most changes are justified!**

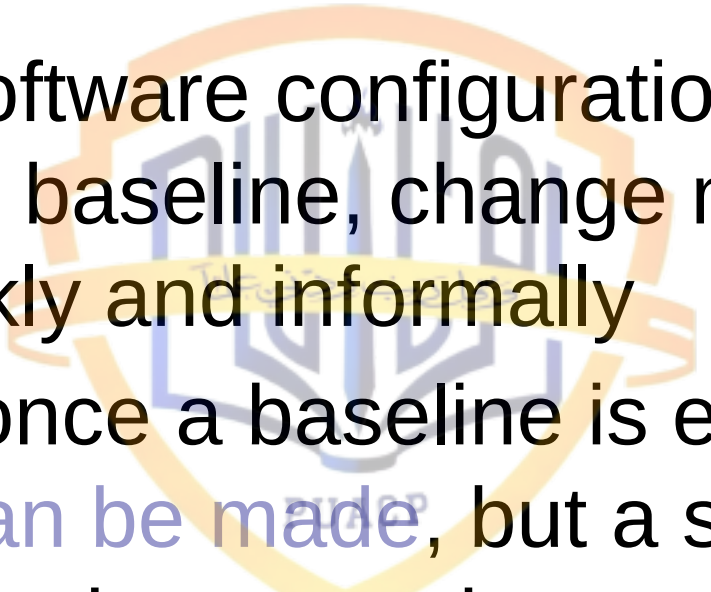
- 
- 
- We, in the software industry need to be ready to manage and implement changes in configuration items
 - We need to have mechanisms in place to manage changes in all work products, in all processes of software development
 - We can apply the concept of baselines

How to Manage Change?

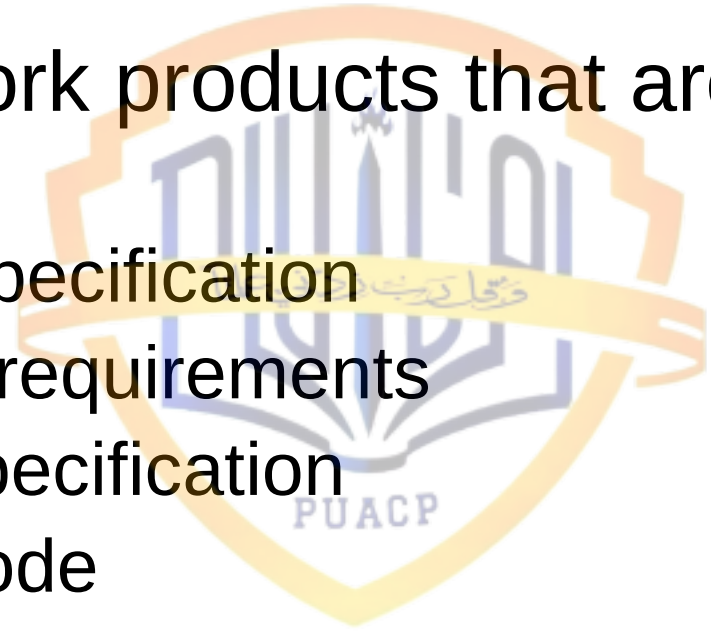
- A baseline is a software configuration management concept that helps us to control change without seriously impeding justifiable change

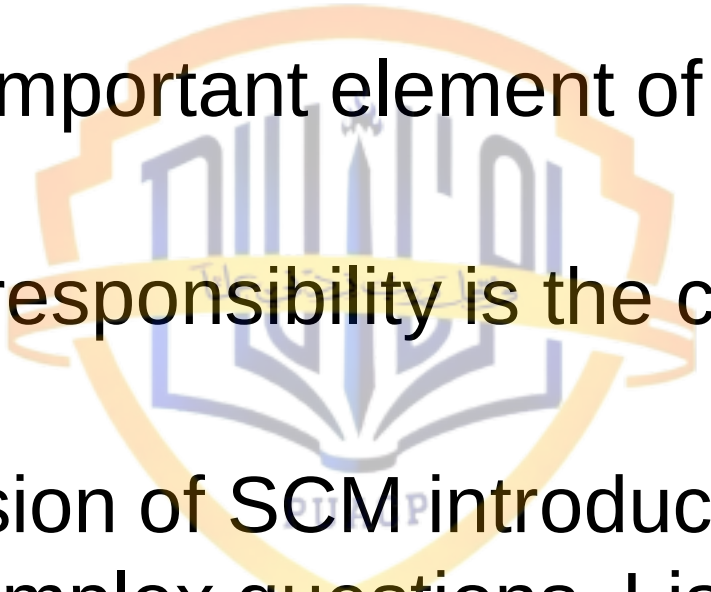
Baseline

- A specification or product that has been formally reviewed and agreed upon, that thereafter serves as the basis for further development, and that can be changed only through formal change control procedures
 - IEEE Std. No. 610.12-1990

- 
- 
- Before a software configuration item becomes a baseline, change may be made quickly and informally
 - However, once a baseline is established, changes can be made, but a specific, formal procedure must be applied to evaluate and verify each change request

- 
- In the context of software engineering, a baseline is a milestone in the development of software that is marked by the delivery of one or more software configuration items and approval of these software configuration items is obtained through a formal technical review or inspection

- 
- 
- Typical, work products that are base-lined are
 - System specification
 - Software requirements
 - Design specification
 - Source code
 - Test plans/procedures/data
 - Operational system

- 
- 
- SCM is an important element of SQA program
 - Its primary responsibility is the control of change
 - Any discussion of SCM introduces a set of following complex questions. Listen carefully to these questions. Each one of them is related to one or more aspect of software quality

SCM Questions - 1

- How does an organization identify and manage the many existing versions of a program (and its documentation) in a manner that will enable change to be accommodated efficiently?
- How does an organization control changes before and after software is released to a customer?

SCM Questions - 2

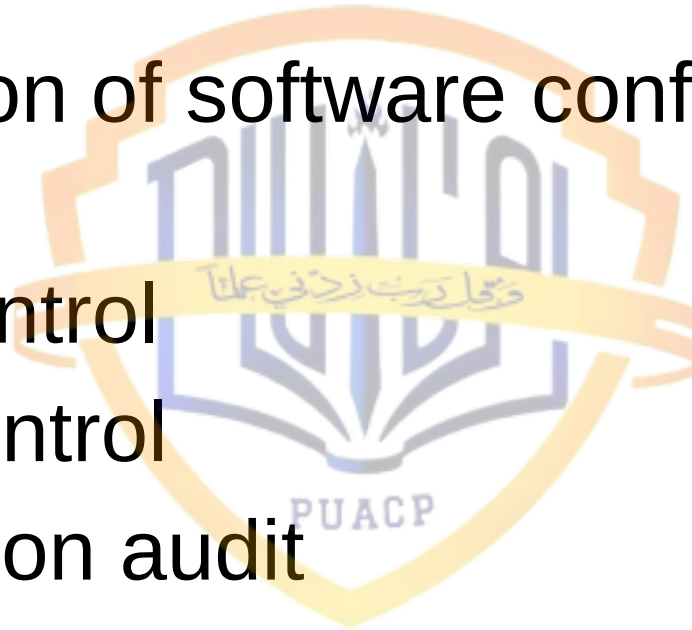
- Who has the responsibility for approving and ranking changes?
- How can we ensure that changes have been made properly?
- What mechanism is used to appraise others of changes that are made?

- These questions lead us to definition of five SCM functions



SCM Functions

- Identification of software configuration items
- Version control
- Change control
- Configuration audit
- Status accounting/reporting



SCM Function: Identification

- To control and manage software configuration items, each item must be separately named or numbered
- The identification scheme is documented in the software configuration management plan
- The unique identification of each SCI helps in organization and retrieval of configuration items

SCM Function: Version Control - 1

- Version control combines procedures and tools to manage different versions of configuration items that are created during the software process
- The naming scheme for SCIs should incorporate the version number

SCM Function: Version Control - 2

- Configuration management allows a user to specify alternative configurations of the software system through the selection of appropriate versions
- This is supported by associating attributes with each software version, and then allowing a configuration to be specified [and constructed] by describing the set of attributes

SCM Function: Change Control - 1

- Change control is vital
- Too much change control and we create problems. Too little, and we create other problems (elaborate this point more)
- The art of progress is to preserve order amid change and to preserve change amid order
 - Alfred North Whitehead

SCM Function: Change Control - 2

- For large projects, uncontrolled change rapidly leads to chaos
- For medium to large projects, change control combines human procedures and automated tools to provide a mechanism for the control of change

SCM Function: Configuration Audit

- 1

- How can we ensure that the approved changes have been implemented?
 - Formal technical reviews/inspections
 - Focuses on the technical correctness of the modified item. The reviewers/inspectors assess the SCI to determine consistency with other SCIs, omissions, or potential side effects
 - Software configuration audit

SCM Function: Configuration Audit

- 2

- A software configuration audit complements the formal technical reviews/inspections by assessing a configuration item for characteristics that are generally not considered during review
- The SCM audit is conducted by the quality assurance group

SCM Function: Status Accounting/Reporting

- The status accounting function provides a corporate memory of project events that supports accomplishment of other configuration management items
 - What happened?
 - Who did it?
 - When did it happen?
 - What else will be affected?

Summary

- In today's lecture, we have discussed the basic concepts of software configuration management
- We'll continue our discussion on this topics and we'll explore issues related to software configuration management in the next few lectures

References

- Software Engineering: A Practitioner's Approach by Roger S. Pressman (Chapter 9.1-9.2)

